Lecture 20: Decision Trees

CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

Announcements

- PA5, WA5 due Sunday April 12

A note on validation

- After we find the best hyperparameters by training multiple models on the training data (that doesn't include the validation set) and testing on the validation set, we create a final model by training on the "full training" set (training+validation). Then we test with testing data
- We didn't specify this before.
- The Week 10 notebook also overlooked this. This has now been fixed.

Cross-validation

The main purposes of validation

are model selection (hyperparameter search)

and performance estimation (predicting how good the model will be on test data).

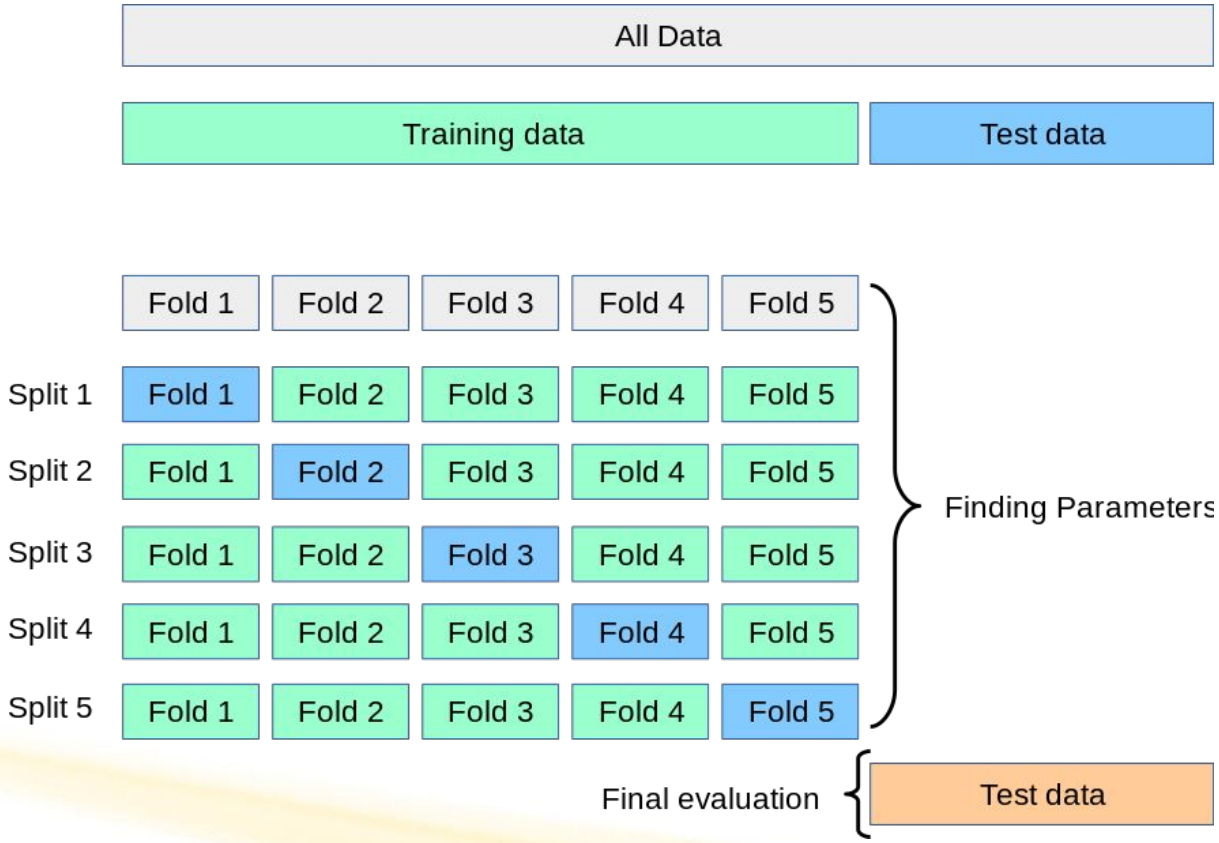
Cross-validation is a generalization of validation.

Types of Cross-Validation

- K-Fold Cross-Validation
- Leave-One-Out Cross-Validation (LOOCV)
- Stratified Cross-Validation
- Many more…

K-Fold Cross-Validation

Example for k=5:



Dataset Splitting: Data is divided into subsets, called **folds**.

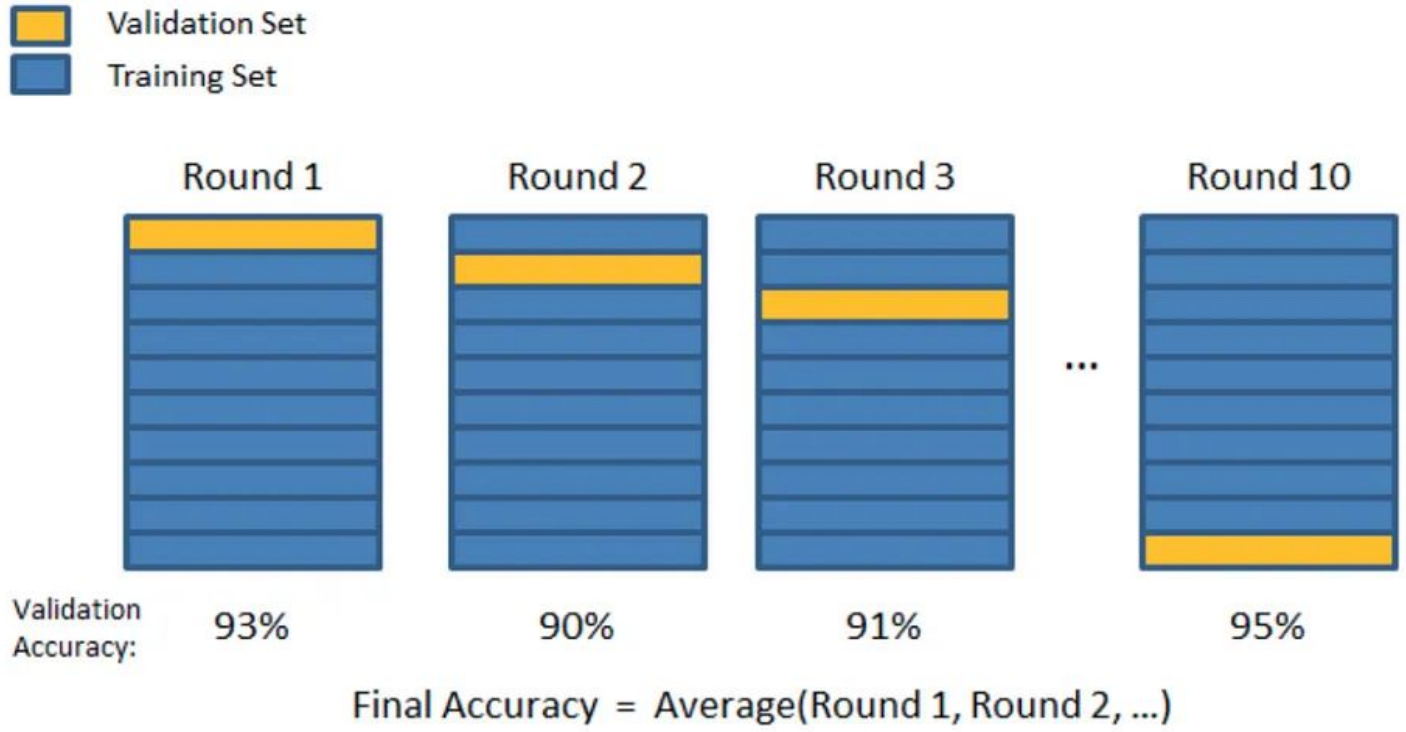
Training and Validation: Model is trained and validated on different folds. One fold used as validation set, others for training.

Repetition: Process repeats with each fold as the validation set.

Performance Assessment: Evaluates models across all folds. Ensures the model generalizes well to new data (avoid overfitting).

K-fold cross validation example for k=10

- Divide the dataset into k roughly equal-sized subsets (called **folds**)
- Train the model on k-1 of these folds, and test on the remaining one.
- Repeat k times, with each fold serving as the validation set exactly once.



How do we choose the number of folds K?

Choice of k is semi-arbitrary.

- Larger k:
  - More computationally expensive, more models needed.
  - Usually less bias, but higher variance
  - Fewer combinations possible

Common choices:

- k = 5 folds
- **k = 10 folds** → most common
  - Usually a good balance of computational cost and bias

You can also try out different values of k and compare, but can be time/resource intensive.

Algorithm for performance estimation:

- Break the data into *k* folds
- Train a model on *k*–1 of these folds, and test on the remaining one.
- Performance metrics like accuracy are averaged across all iterations to obtain a final estimate. You can also think of these as producing a range of potential accuracy values: $[a_{min}; a_{max}]$
- The full data (all *k* subsets taken together) is trained on for a final model
- The final model is tested with test data

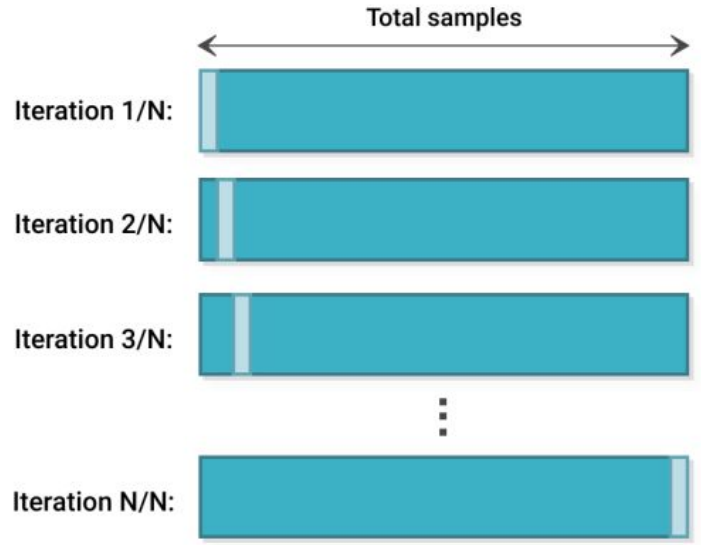
Algorithm for model selection:

- Break the data into k folds
- For each algorithm (e.g. Decision Trees; 3NN; 5NN):
  - Train a model on k–1 of these folds, and test on the remaining one.
  - Performance metrics like accuracy are averaged across all iterations to obtain a final estimate.
- Choose the best algorithm (e.g. 3NN or 5NN) based on the average accuracy.
- Train a final model using the best algorithm and the full data (all *k* subsets taken together)
- The final model is tested with test data

Leave-One-Out Cross-Validation (LOOCV)

A special case of k-folds CV where k=n (number of examples).

- A model is created and evaluated for **every example** in the training dataset
  - Each example is held out as the validation set while the model is trained on the remaining data (n-1 examples).
  - The trained model is evaluated with the 1 example validation “set.”
  - This process is repeated for each data point.
- Performance metrics like accuracy are averaged across all iterations to obtain a final estimate.



Leave-One-Out Cross-Validation (LOOCV)

LOOCV is computationally very expensive!

- **Don't use on large datasets** (ex. n>1000), or with particularly expensive models

Useful for small datasets (ex. n<1000) because provides a very reliable estimate of model performance.

- Every example is given an opportunity to validate
- **Use when it's critical to get an accurate estimate of model performance**

Leave-One-Out Cross-Validation (LOOCV)

Algorithm

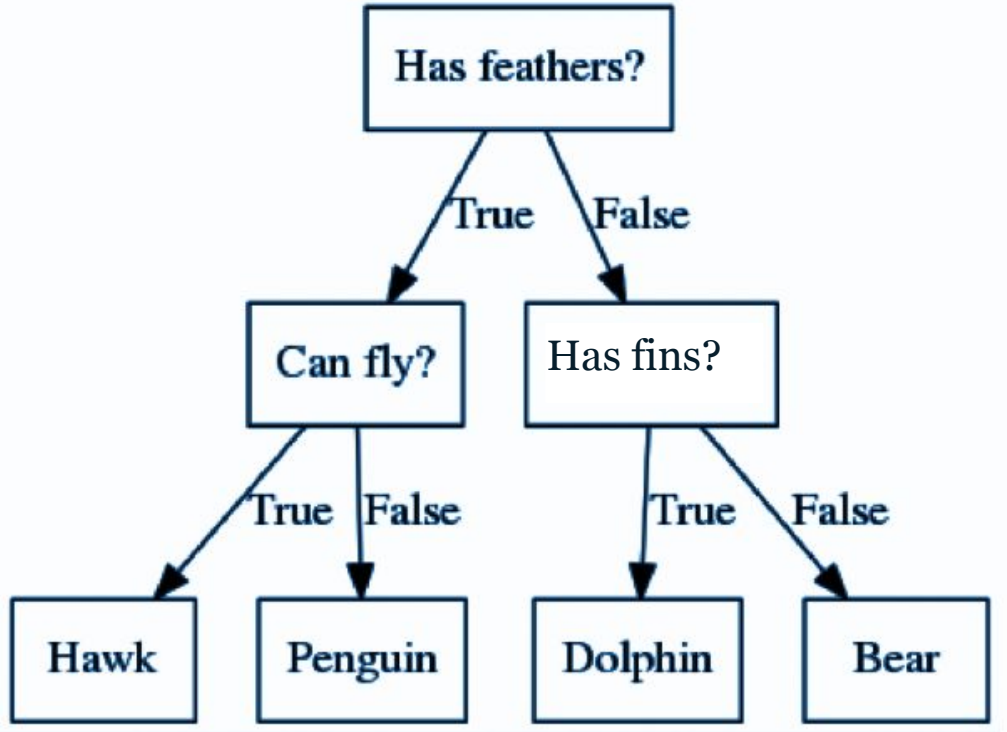
1. Split a training dataset into a training set and a validation set.
   - Use 1 example as the validation set.
   - Use the other n-1 examples as the training set.
     - You leave one observation “out” from the training set.
     - This is where the method gets the name “leave-one-out” cross-validation.
2. Train a model on the n-1 training examples
3. Evaluate the model.
4. Repeat n times, with each data point being used as the test set exactly once.
5. Evaluate across all iterations.

Decision Trees

Decision Trees (DT)

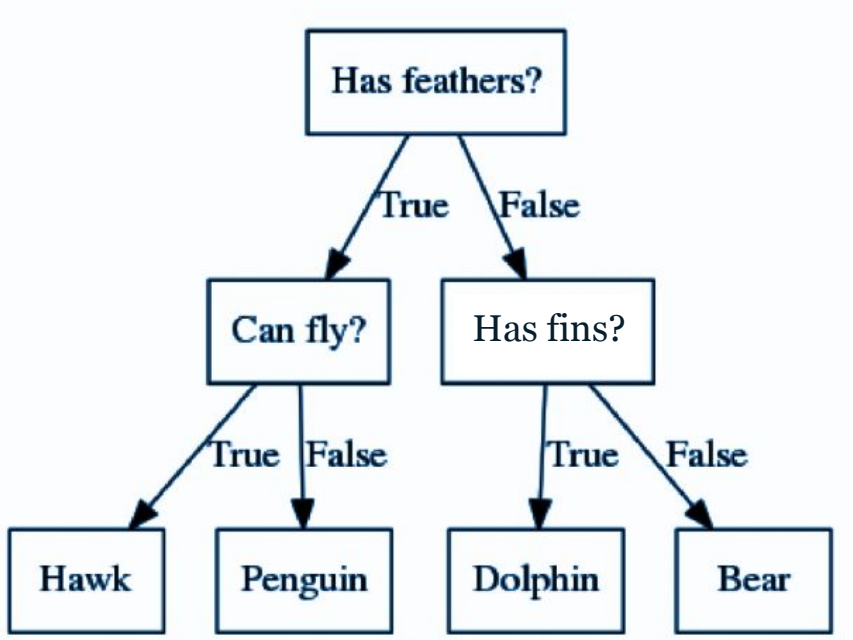
- Define a hierarchy of rules to make a prediction
  - Root and internal nodes test rules.
  - Leaf nodes make predictions
- DT learning is about **learning such a tree** from **labeled** training data
  - Supervised learning algorithm

ID	Has feathers?	Can Fly?	Has Finns?	Class Label
7	True	False	False	*Penguin*



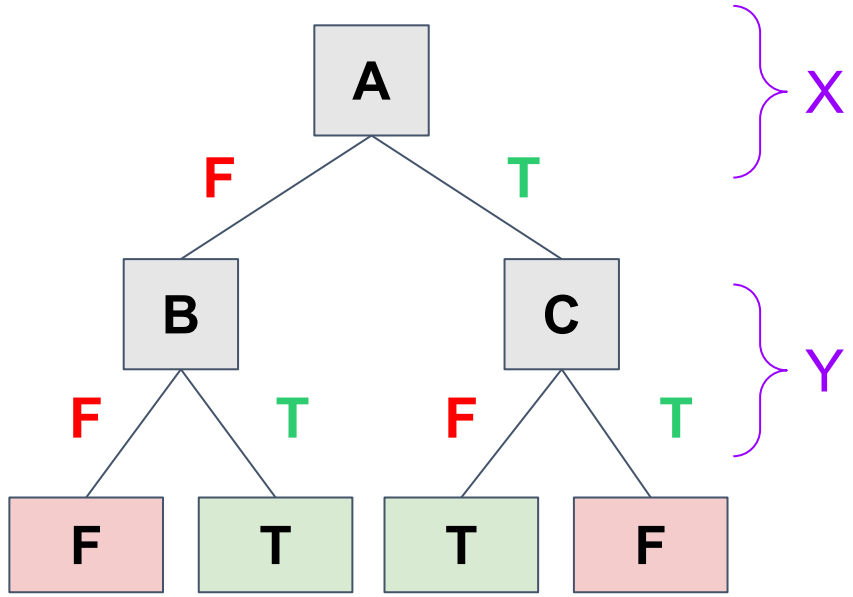
Decision Trees

- DTs partition the space of possibilities.
- Each feature can lead to a decision point
- Each answer comes from the feature value for that particular example/observation
- Decisions need to be examined **in order**
- May not need all the features
    - <u>2 meanings</u>:
        1. On any particular path, we may not need all the features
            - Ex. True to “Has feathers” means we don’t need “Has fins”
        2. We may not need a particular feature at all
            - Ex. We don’t need “Has scales?” for this particular animal classification



Expressiveness of Decision Trees

- As **Boolean functions**, DTs can ***represent any*** **Boolean function**.
  - Ex. (A ∩ B) U (A ∩ C)
- Can be rewritten as rules in Disjunctive Normal Form (DNF)
  - XOR - simply an OR of a bunch of ANDs
  - True when one of the operands is true
  - **A truth table row = the path to a leaf**
- Powerfully expressive and easy to understand

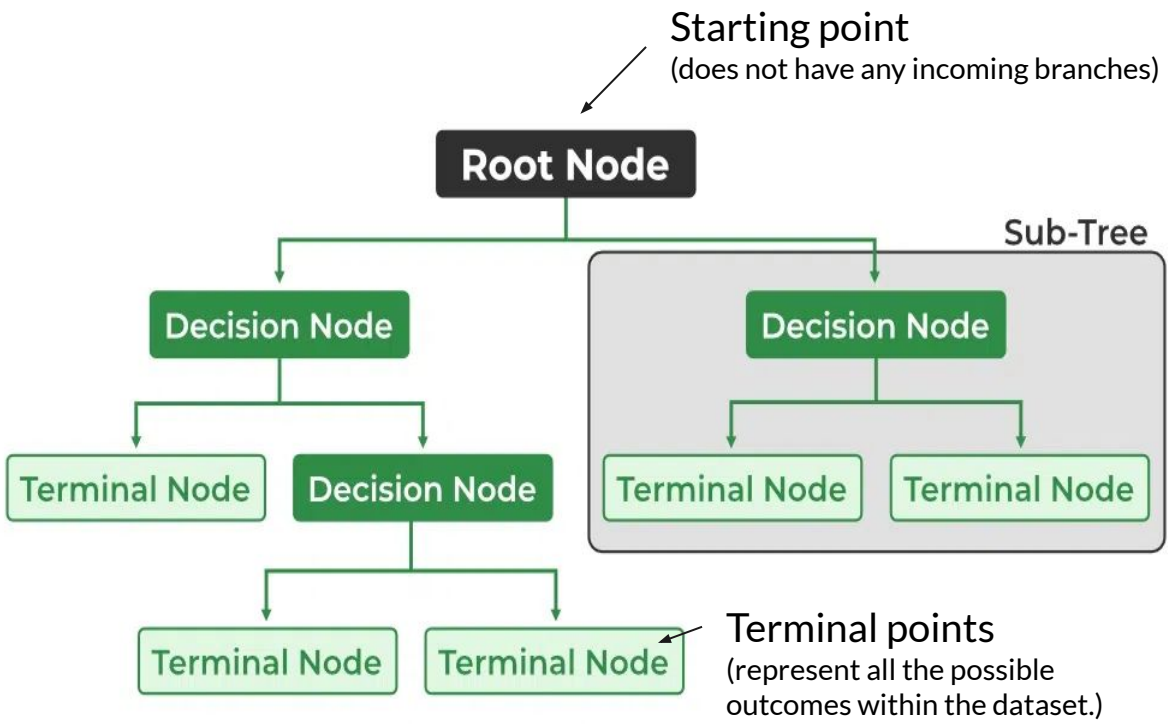


X	Y	X xor Y
F	F	F
F	T	T
T	F	T
T	T	F

Advantages of Decision Trees

- **Powerful Representation**: Can compactly represent complex concepts.
- **Expressiveness**: Can represent *any* boolean function
- **Variable-Sized Hypothesis Space** (H): Can explore a broad range of possibilities.
- **Intuitive Decision-Making**: Mirror common forms of human decision-making (ex. game of questions).
- **Flexible**: Can handle non-linear data and does not assume normality

Decision Tree Terminology



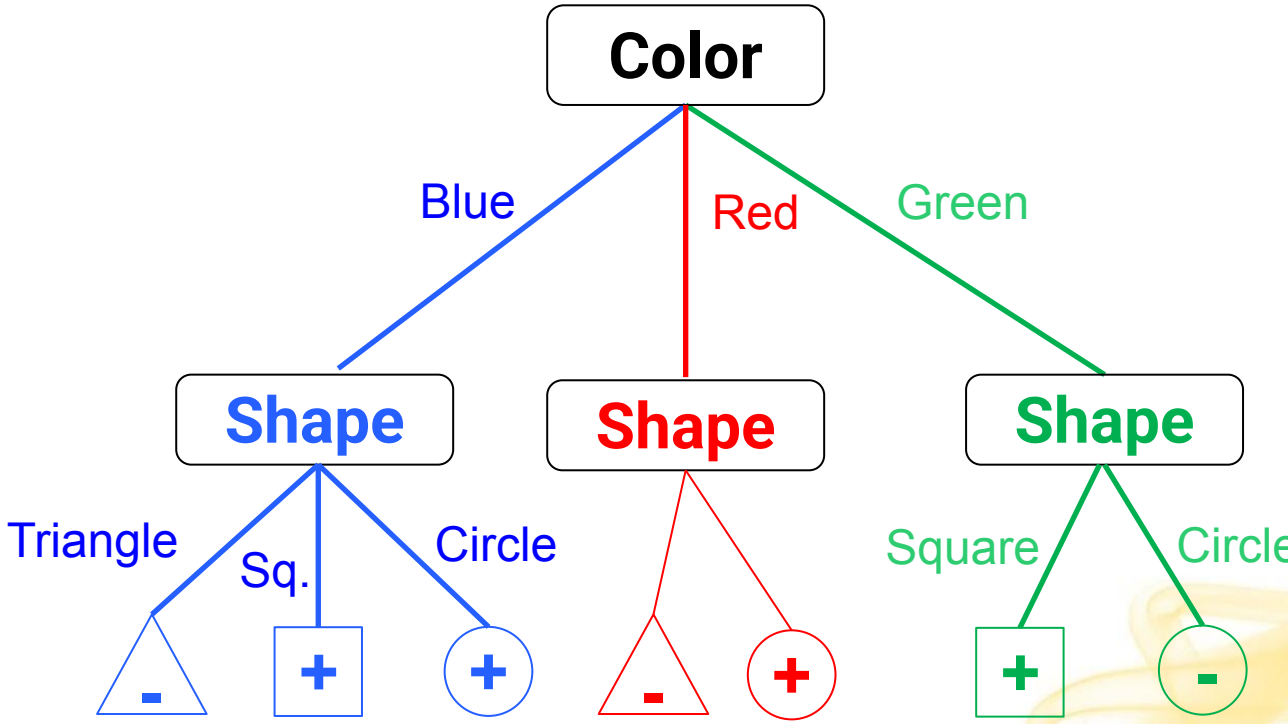
A binary decision tree (BST) is a decision tree in which each node has two branches.

Parts of a decision tree:

- **Root node**: the first/initial decision node at the top of the tree. It represents the starting point for decision-making.
- **Decision node**: where a specific feature of the data is tested. Based on the value of this feature, the tree decides which branch to follow.
- **Branch**: a connection between decision nodes in the tree. Each branch represents a possible outcome or decision based on the feature being tested at the decision node.
- **Layer/Level**: all of the nodes that are the same distance from the root node.
- **Leaf/Terminal node**: a node that does not test a feature and so is where a decision is made.
- **Height**: the number of layers in a decision tree (excluding root).
- **Path**: the route from the root node to a given leaf

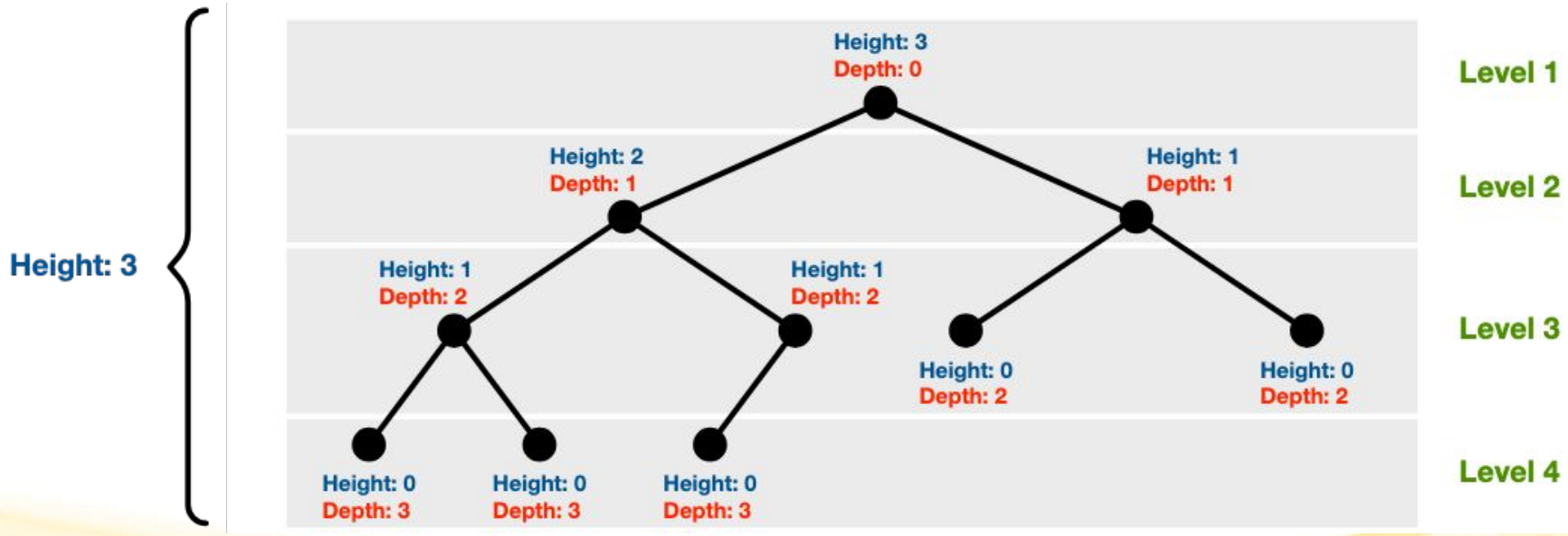
Decision Tree Example

- How many dimensions?
  - 2: color and shape
- How many possible values for each dim?
  - 3 values for each:
    - color: (red, blue, green)
    - shape: (triangle, square, circle)
- How many class labels/predictions?
  - 2: (+) and (-)
- How many possible paths?
  - 7: (red, triangle), (red, circle), (blue, square)...
- How many leaf nodes?
  - 7: 3 for blue path, 2 for red path, 2 for green path



Decision Tree Depth vs Height

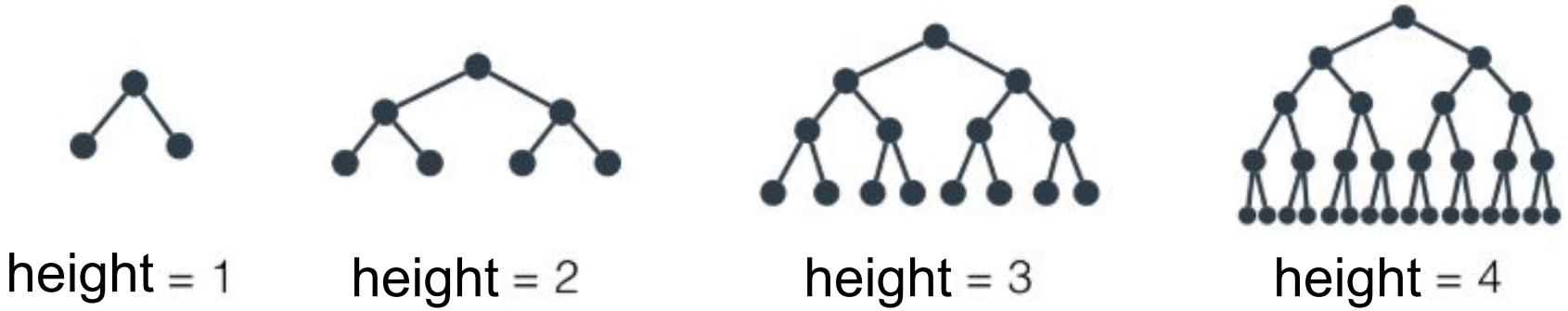
- **Depth** d = length of the path from root node to a given leaf node
- **Height** h = maximum depth to any leaf node – the number of layers/levels
  - Binary DT (balanced): $h = \log_2(n)$



Decision Tree Depth and Hypothesis Space

As height increases, so does the hypothesis space and complexity.

- Affects bias and variance - more on that in a bit



Ex. Balanced Binary Decision Tree

Maximum depth of a decision tree

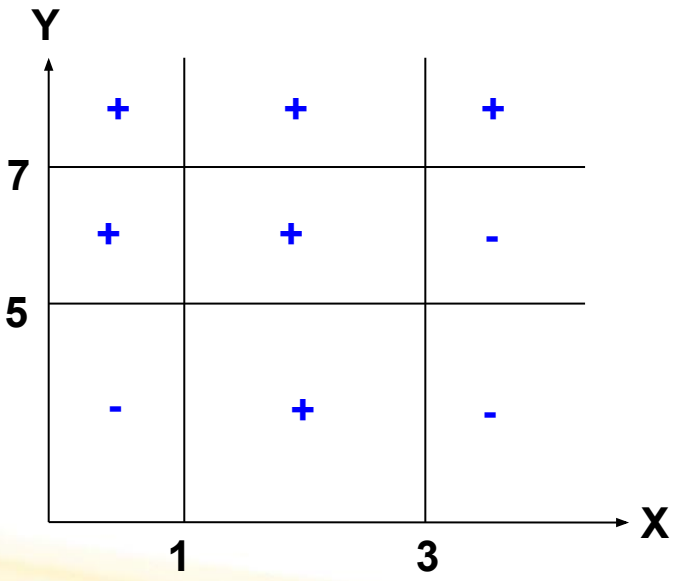
Output of Decision Trees

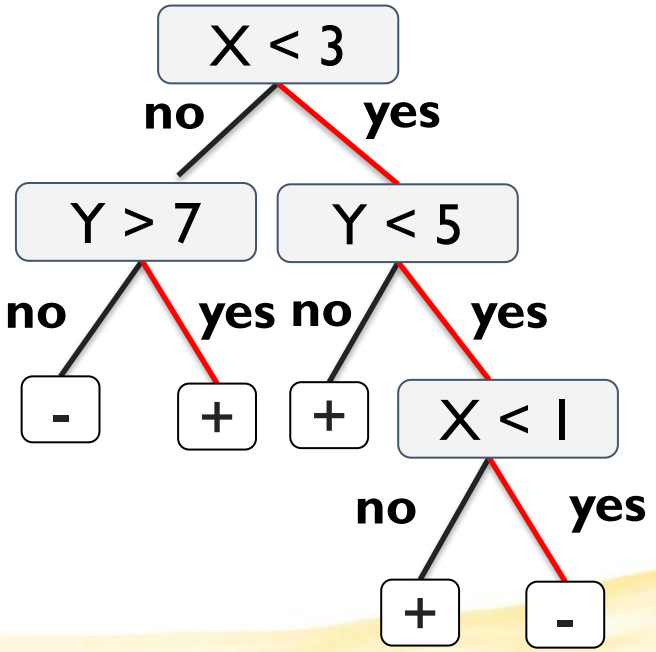
Output is a **prediction**

- For classification: a **discrete category**
  - Most common use case, and what we'll look at
- For regression: a **real number**

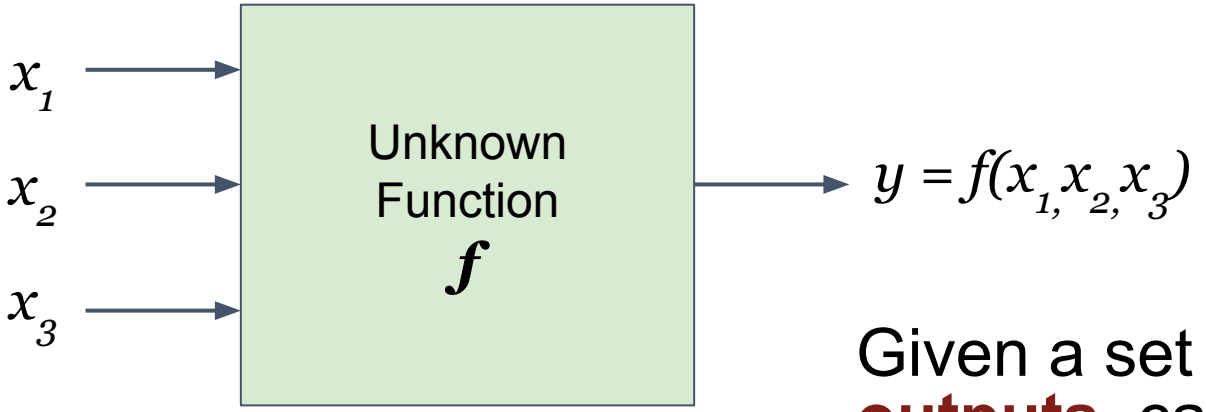
Decision Boundaries

- Numerical values can be used either by discretizing or by using thresholds for splitting nodes
- DT divides the feature space into axis-parallel rectangles, each labeled with one of the labels
- Example:





Learning Problem Example



	x_1	x_2	x_3	y
1	0	0	1	0
2	0	1	0	0
3	1	0	0	0
4	0	1	1	1
5	1	1	0	1
6	1	0	1	1
7	1	1	1	0

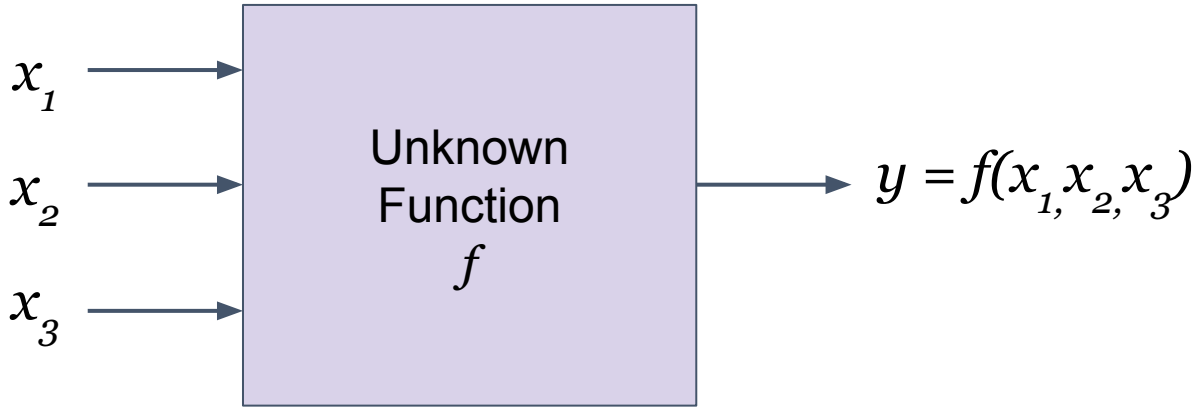
Given a set of **known inputs** and **outputs**, can we learn an **unknown function**?

Goal: **Find the function** that best divides the feature space, so that we can predict the output for a previously unseen set of inputs.

Learning Decision Trees with Supervision

The basic idea is very simple:

- Recursively or iteratively partition the training data into homogeneous regions (i.e. learn the function that divides the feature space)
- Within each group, fit a simple supervised learner (i.e. predict the majority label)

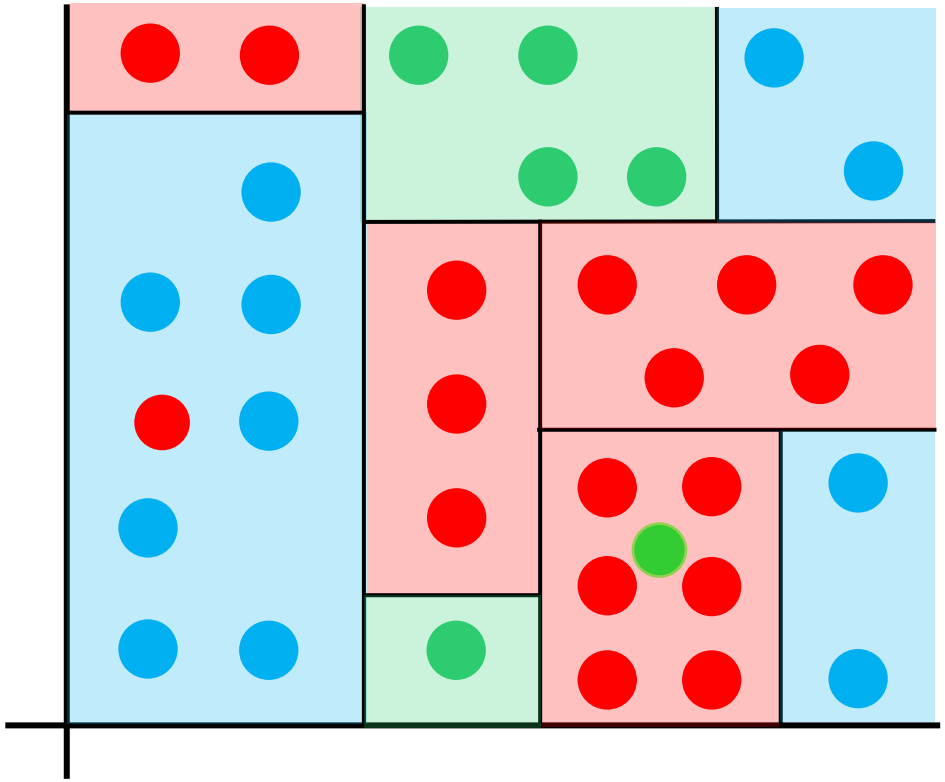


	x_1	x_2	x_3	y
1	0	0	1	0
2	0	1	0	0
3	1	0	0	0
4	0	1	1	1
5	1	1	0	1
6	1	0	1	1
7	1	1	1	0

Learning Decision Trees with Supervision

What do you mean by "homogeneous" regions?

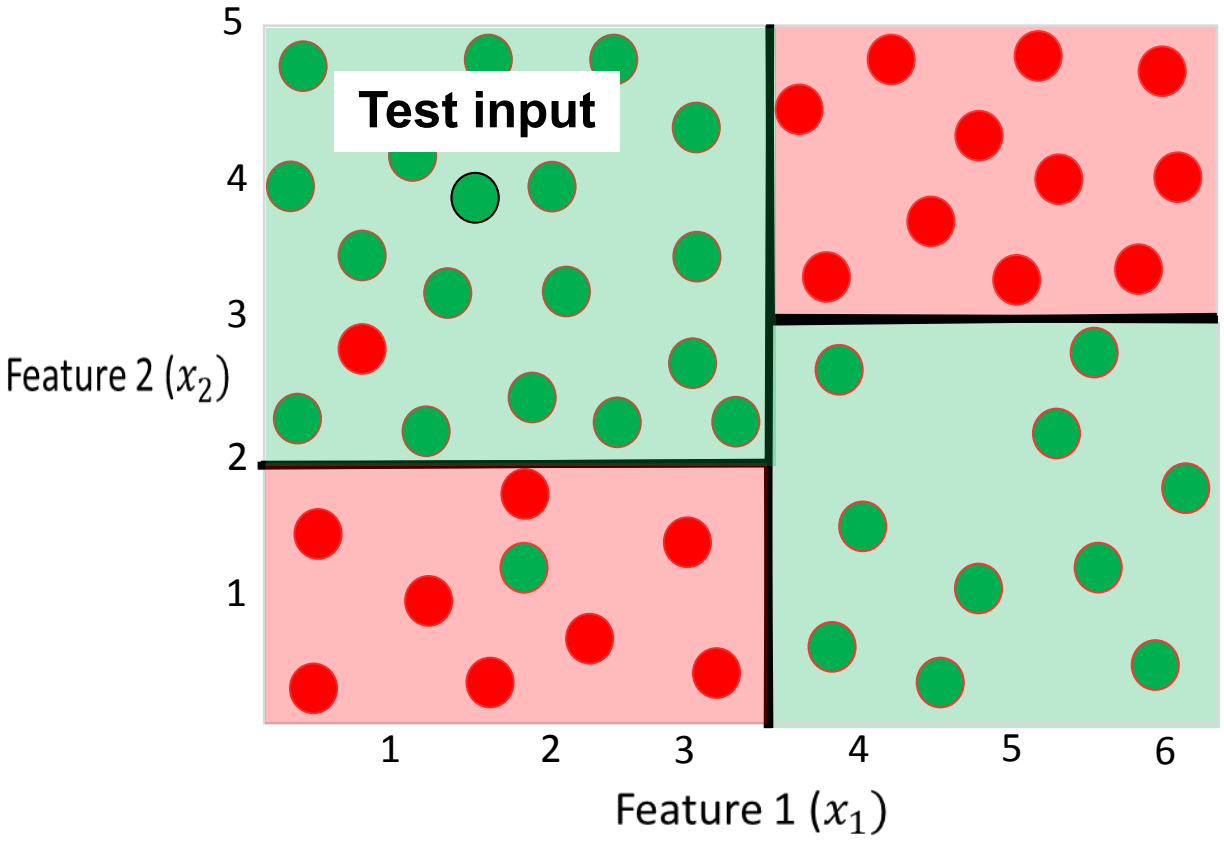
A homogeneous region will have all (or a majority of) training inputs with the same/similar outputs

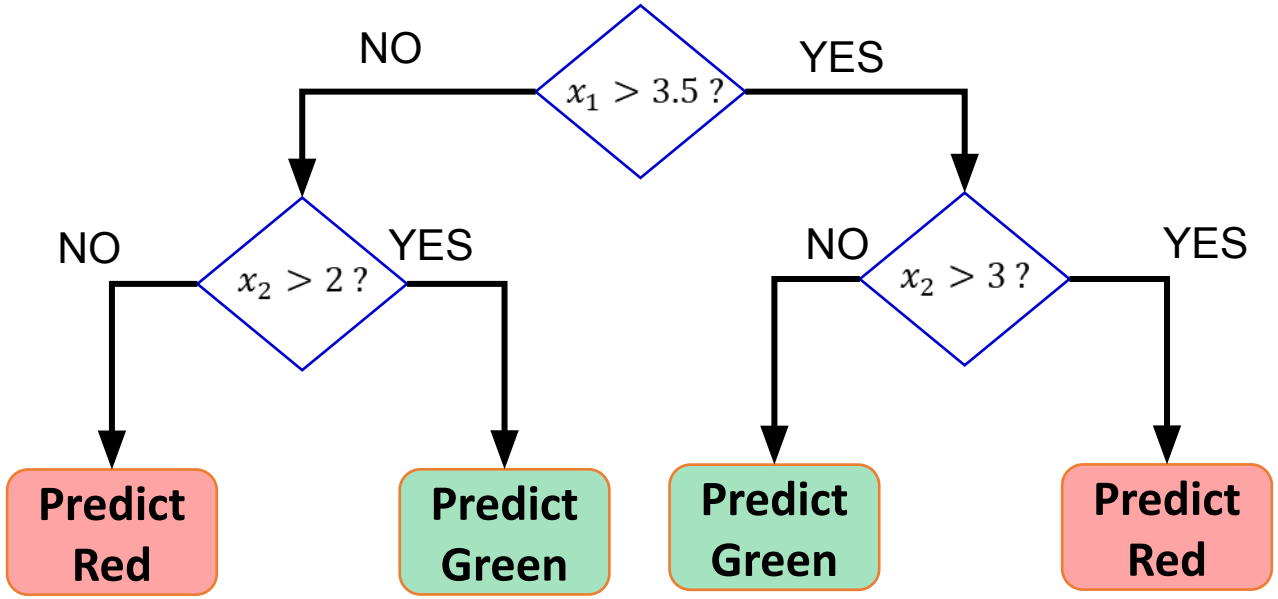


Even though the rule within each group is simple, we are able to learn a fairly sophisticated model overall.

In this example, each rule is a simple horizontal/vertical classifier but the overall decision boundary is rather sophisticated)

Decision Trees for Classification

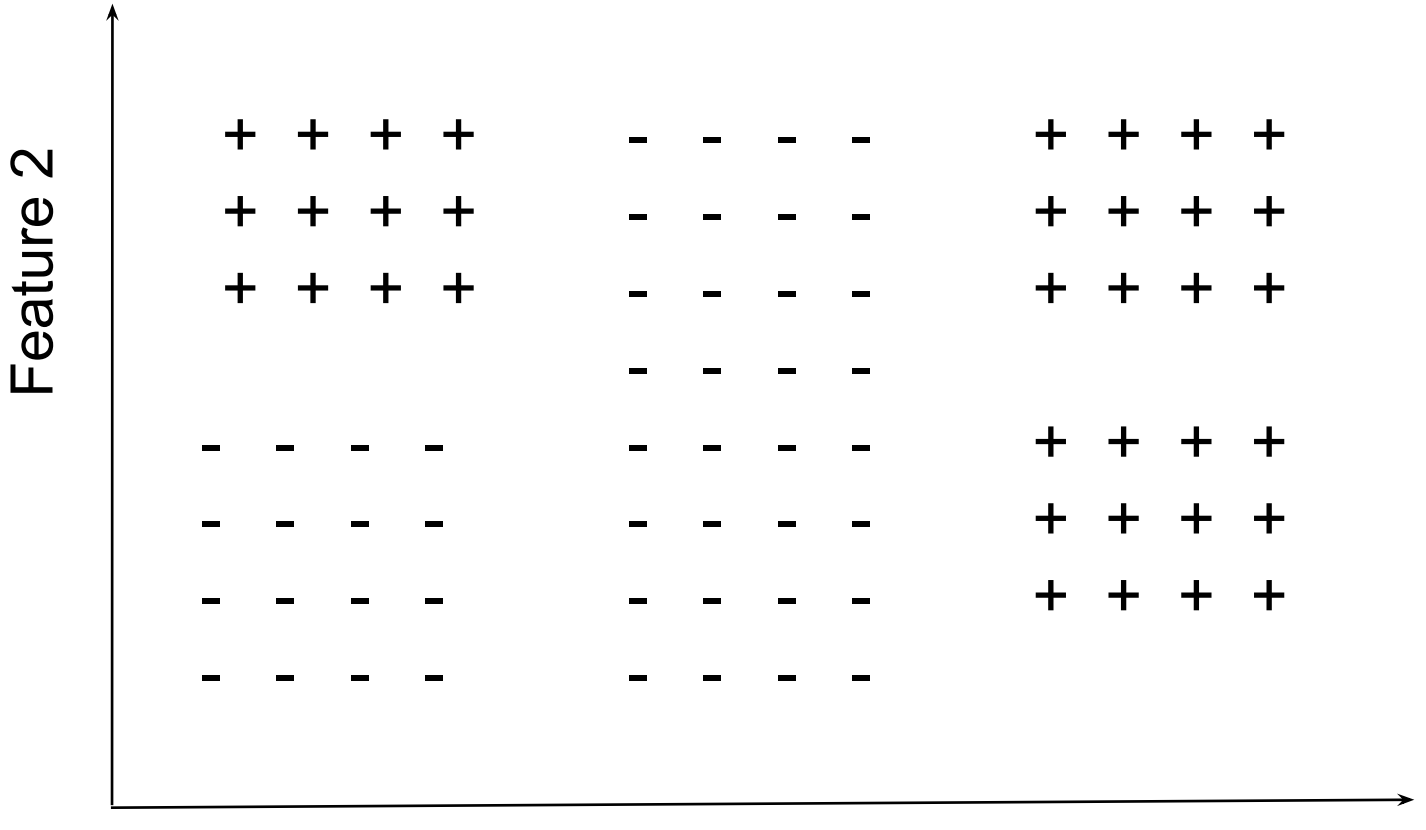




DT is very efficient at test time: DT here predicts the label by doing just 2 feature-value comparisons!

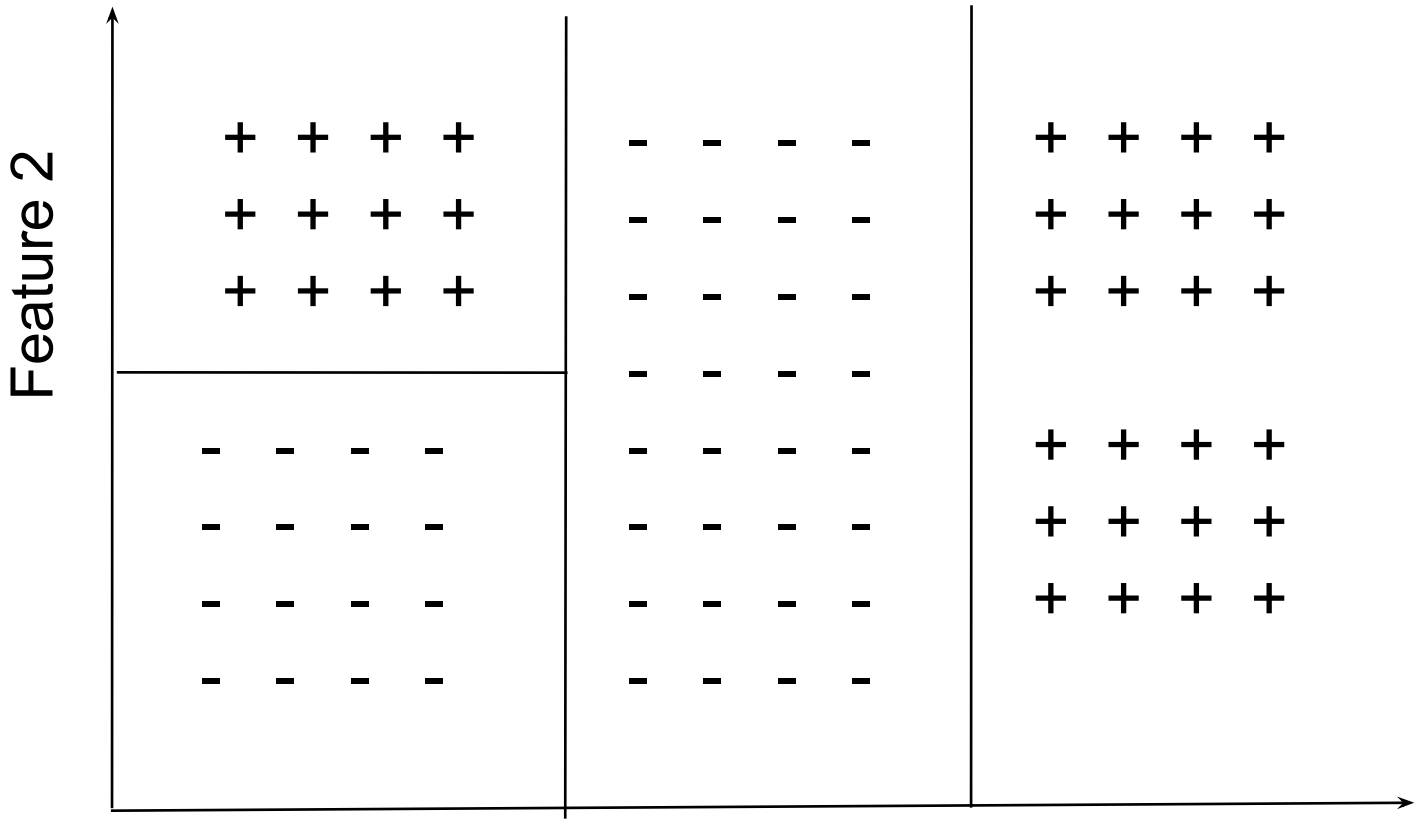
Remember: Root node contains all testing inputs
Each leaf node receives a subset of testing inputs

Decision Tree Structure



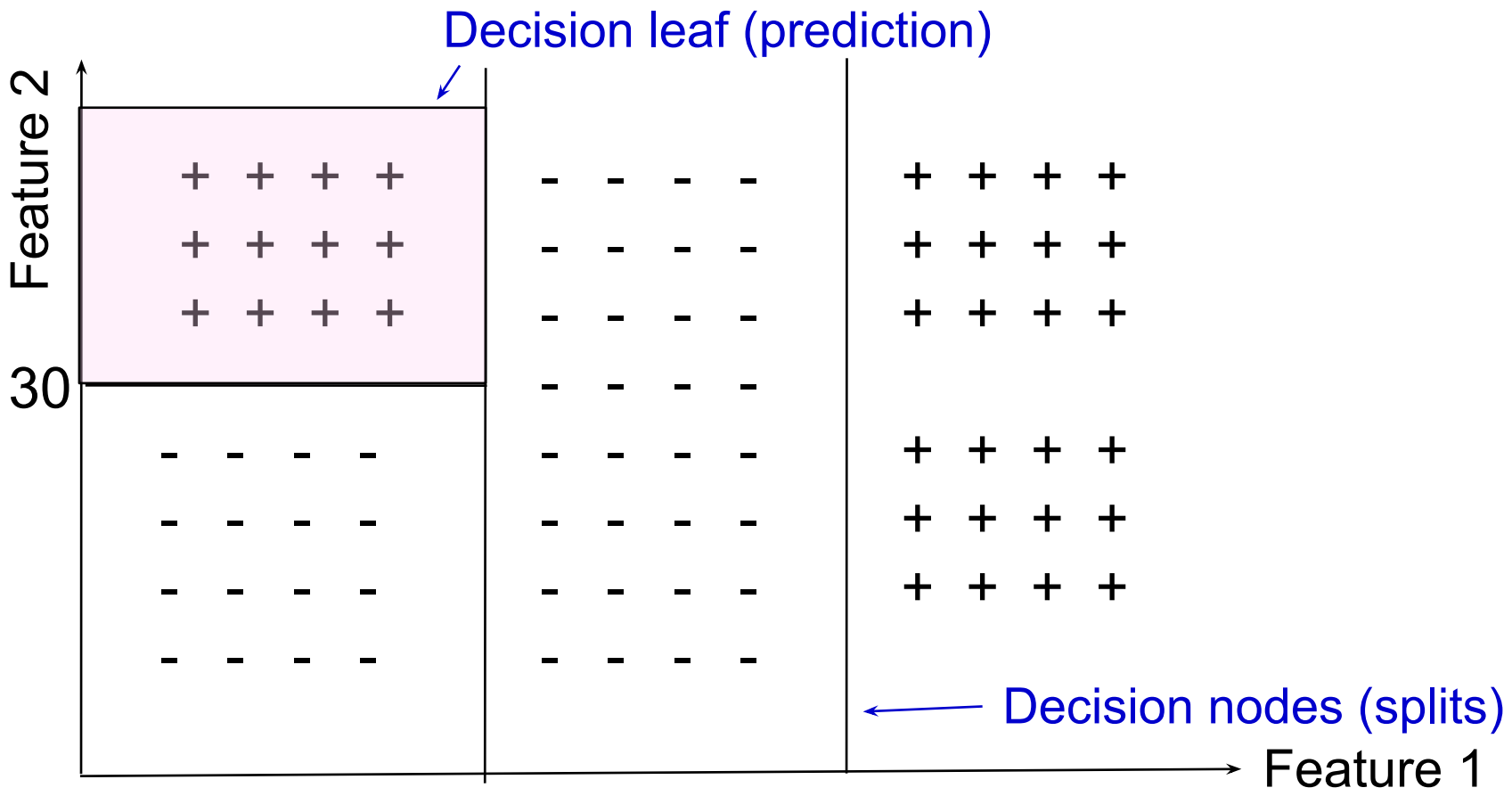
Given a distribution of training examples, draw axis-parallel lines to separate the instances of each class

Decision Tree Structure



Given a distribution of training examples, draw axis-parallel lines to separate the instances of each class

Decision Tree Structure



DTs for Classification: Another Example

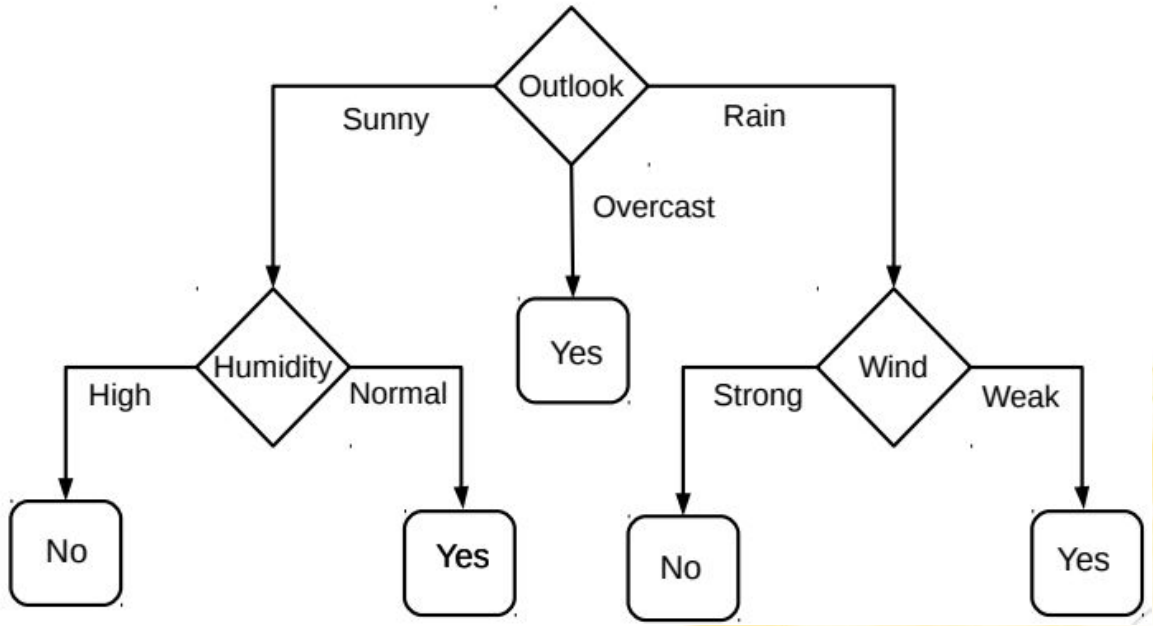
- Deciding whether to play or not to play 🎾 Tennis on a Saturday
- Each input (Saturday) has 4 categorical features: Outlook, Temp, Humidity, Wind
- Binary classification problem: play vs no-play

Training data

day	outlook	temperature	humidity	wind	play
1	sunny	hot	high	weak	no
2	sunny	hot	high	strong	no
3	overcast	hot	high	weak	yes
4	rain	mild	high	weak	yes
5	rain	cool	normal	weak	yes
6	rain	cool	normal	strong	no
7	overcast	cool	normal	strong	yes
8	sunny	mild	high	weak	no
9	sunny	cool	normal	weak	yes
10	rain	mild	normal	weak	yes
11	sunny	mild	normal	strong	yes
12	overcast	mild	high	strong	yes
13	overcast	hot	normal	weak	yes
14	rain	mild	high	strong	no

Example credit: Tom Mitchell

Decision Tree constructed using this data



Decision Tree Size

Goal: make the decision tree as **small/simple** as possible.

- Occam's Razor: the simplest explanation is usually the best one.
- But, finding the provably smallest decision tree consistent with the training data is NP-hard.
  - The algorithm is a search for a simple tree, but cannot guarantee optimality.
- The main decision in the algorithm is the selection of the next feature to partition on.
- Instead of constructing the absolute smallest tree, construct one that is small enough

Decision Tree Algorithms

Many Algorithms:

- ID3 (Iterative Dichotomiser 3) → **We will focus only on this one**
- C4.5 (enhanced ID3)
- Hunt's Algorithm
- CART (Classification and Regression Trees)
- MARS (Multivariate Adaptive Regression Splines)
- SPRINT (Scalable Parallelizable Induction of Decision Tree)
- SLIQ (Supervised Learning In Ques)
- …others

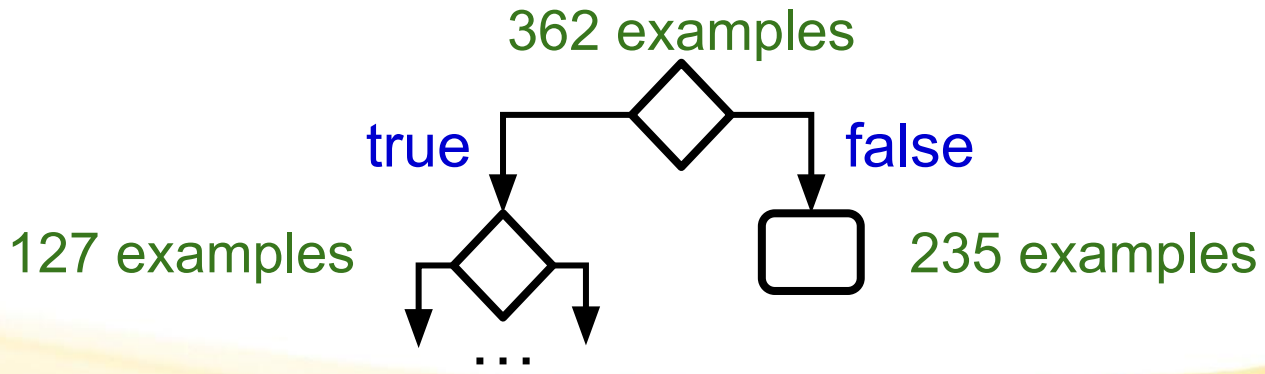
ID3

- ML algorithm that **builds a Decision Tree** from training data
  - Builts it top-down, starting with the root feature node
- Performs a **greedy heuristic search** through the feature space
  - Heuristic = uses known information (i.e. labeled training data) to reduce the search space in order to find a solution quickly
    - Recursively divides the features into two or more groups at each step: hill climbing without backtracking
  - Greedy = favor immediate gains when selecting the next feature to examine
    - maximum information gain (IG) or minimum entropy (H) → will discuss in a moment
- Stops at the smallest acceptable tree
  - Prefer the simplest hypothesis that fits the data (Occam's Razor)
- Resulting tree is used to predict for new, unseen examples
- Can be used both for classification and regression
  - Best for discrete/categorical data

ID3, conceptually

Problem: Given training data, what feature should we examine first?

- Suppose we have a 2-class problem (binary classification):
  - ex. label an output A or B
- And binary input features (each feature has a binary value like true/false)
- When we look at any given feature, it divides the data into 2 parts
  - Those examples for which the feature value is true, and those for which it is false
  - Example:

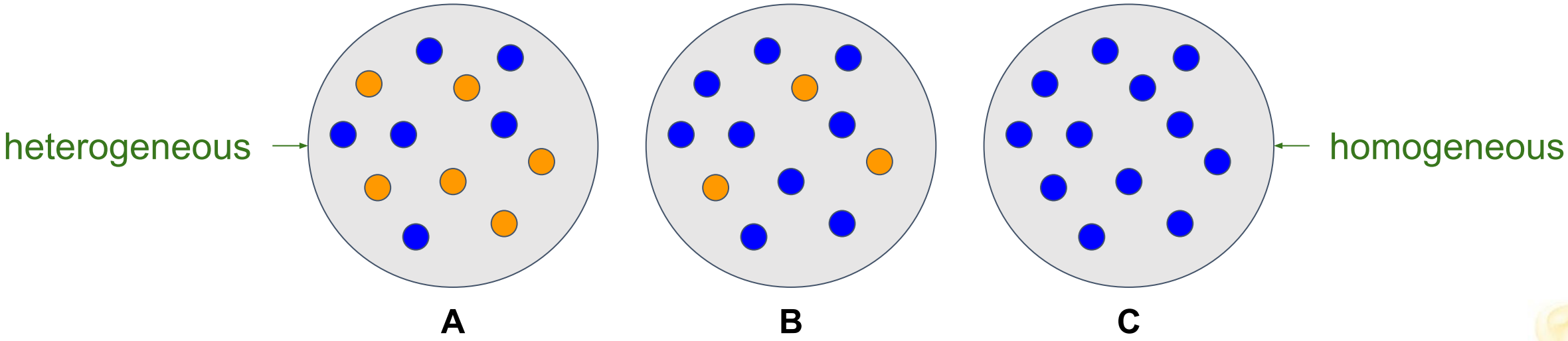


ID3, conceptually

- Each of the 2 subgroups has a different distribution of the class label
  - One group will have x% label A and y% label B, the other will have something else
- We can ask: **Is this a good split**?
  - The *best* split would be one that perfectly determines the output label
    - i.e. if feature value = yes, then label A. if feature value = no then label B.
    - Homogeneous > heterogeneous (mixed) for distribution of output labels in the split
  - But, it's unlikely that just 1 feature will do this.
  - Instead, we can use multiple features to get there.
  - At any given point, we'll use the feature that gets us as close to that ideal as possible
- Recursively split into smaller and smaller groups until stopping criteria is met

Split Criteria Intuition

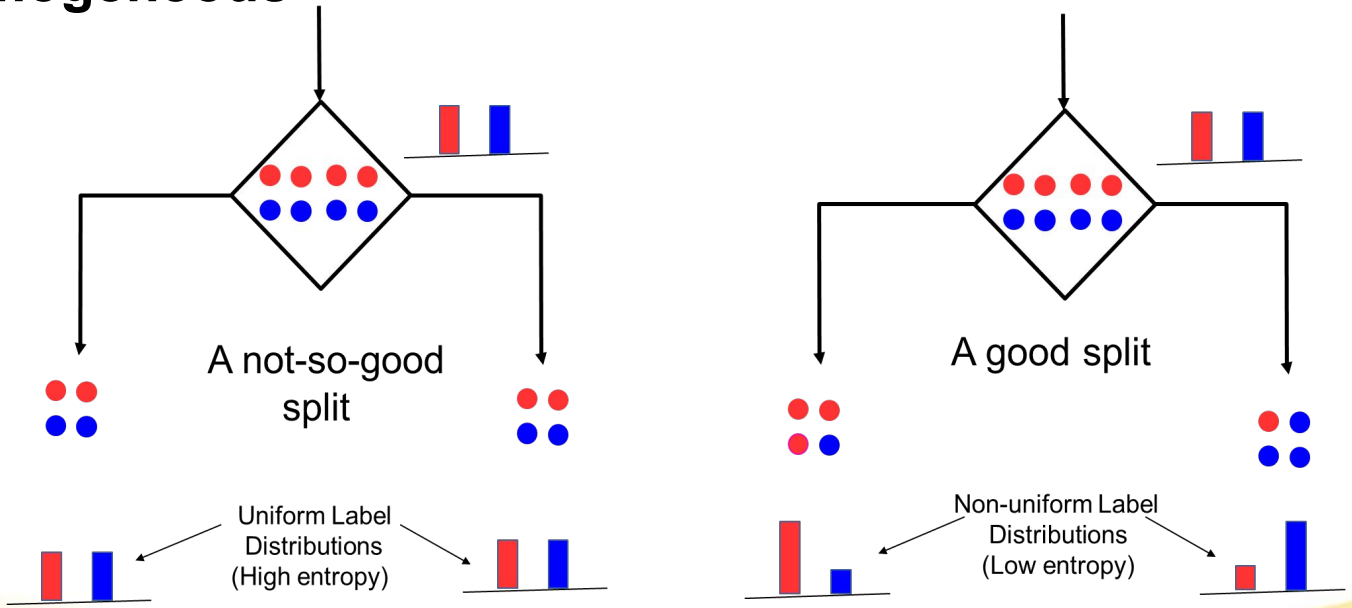
Why is a homogenous split better than a heterogeneous one?



Which of these is easiest to predict/describe?

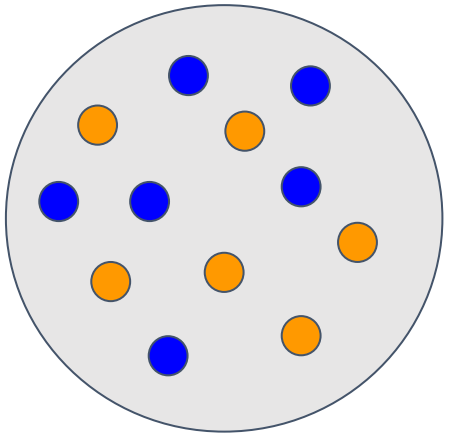
How to Split at Feature Nodes?

- Recall that each internal node receives a subset of all the training inputs
- Regardless of the criterion, the split should result in as "**pure**" groups as possible
  - A pure group means that the majority of the inputs have the **same label/output** - the group is **homogeneous**



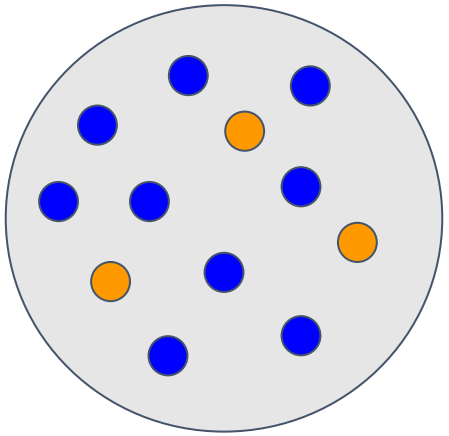
Purity and Impurity

Least Pure
Most Impure

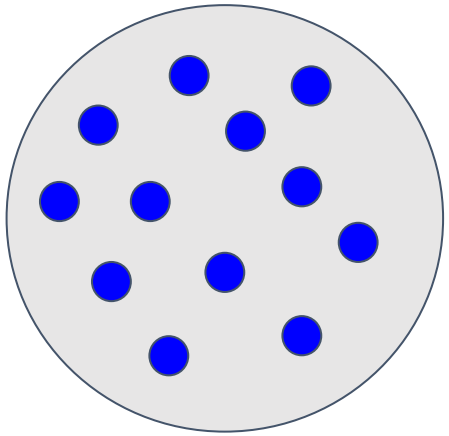


heterogeneous

Less
Pure/Impure

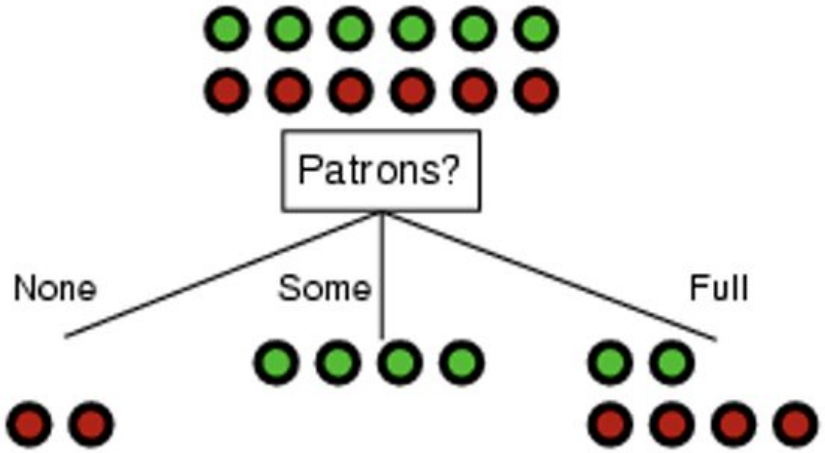


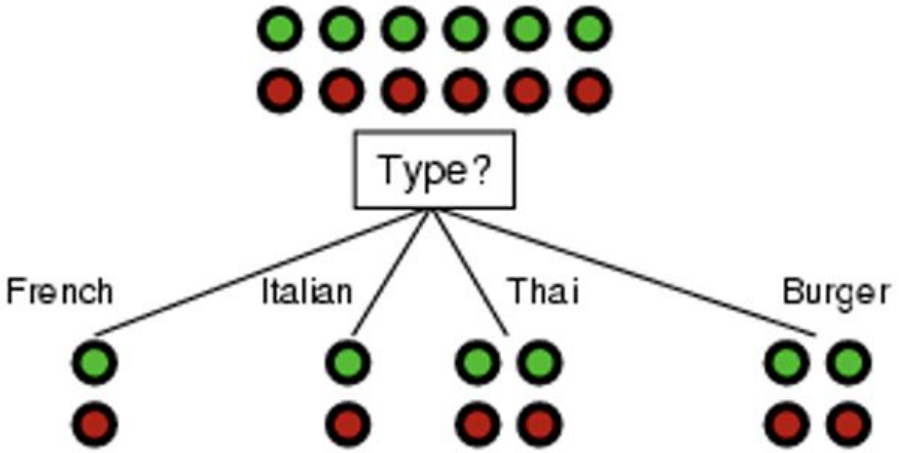
Most Pure
Least Impure



homogeneous

Ex. Which split is better: *Patrons* or *Type*?





Principles of ID3 DT Construction

- We want to form **pure leaves** 🍃
  - Ability to predict the correct classification
  - Deterministic / Homogenous / Pure / High Information Gain is good 👍
  - Uniform distribution / Heterogeneous / Impure / High Entropy is bad 👎
- **Greedy approach** to reach the correct classification
  1. Initially treat the entire dataset as a single box
  2. For each box, choose the split that reduces its entropy/impurity (in terms of class labels) by the maximum amount
  3. Split the box to achieve highest reduction in impurity
  4. Return to Step 2
  5. Stop when all boxes are pure

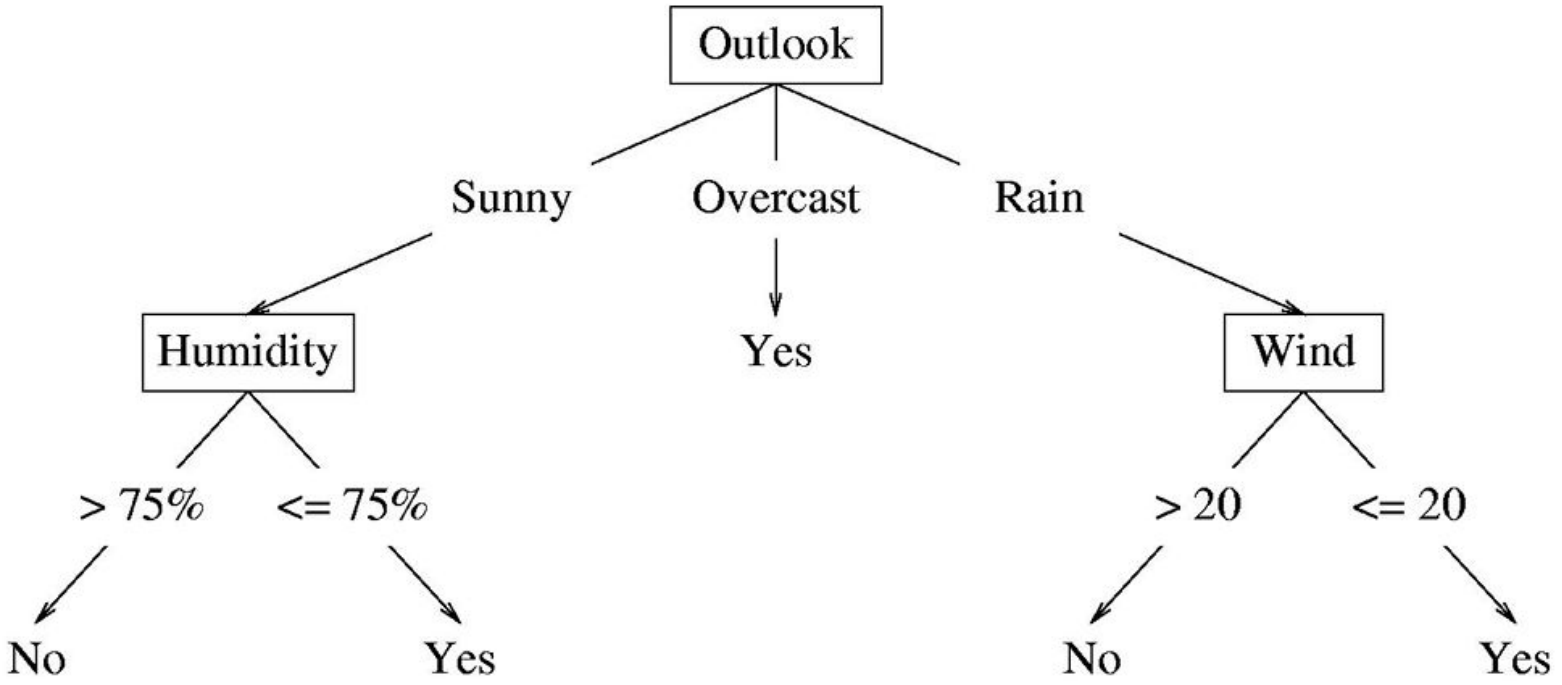
The Goal

Remember:
The goal is to find the **next best feature** for splitting the dataset (greedy).

The “best” feature is the one that **minimizes entropy or maximizes information gain**, thereby reducing uncertainty and making the dataset more homogeneous (in terms of the target variable or class labels) after the split, in order to achieve better classification performance in Decision Tree algorithms.

What if features are continuous?

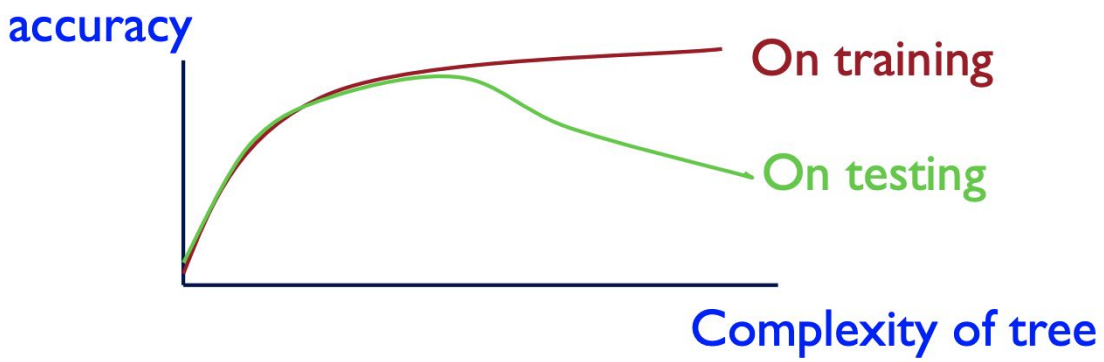
Internal nodes can test the value of a feature against a threshold.

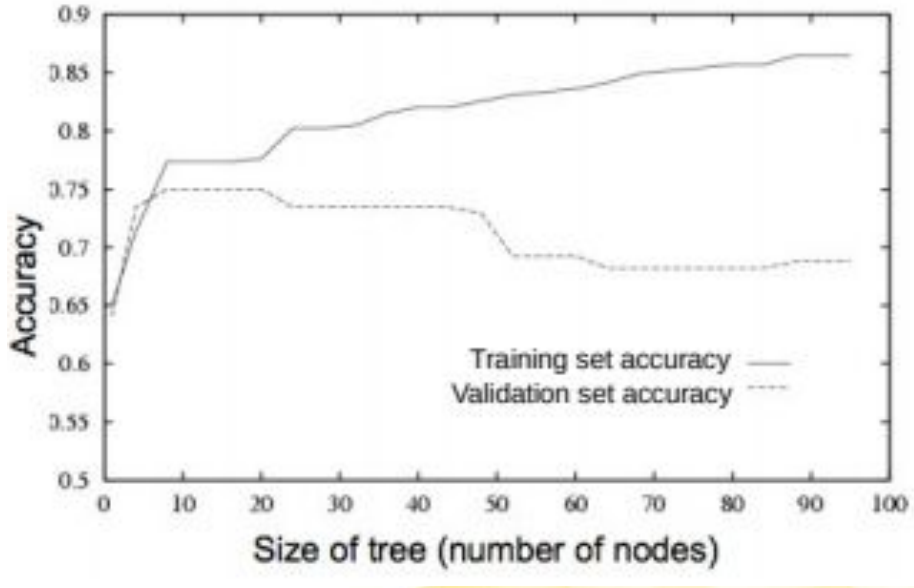


When to stop growing a Decision Tree

Stop expanding a node further (make it a leaf node) when:

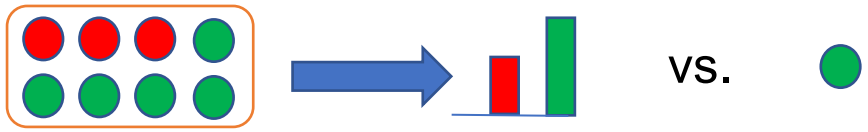
- All training examples in the node have the same label (the node becomes **pure**)
- There are **no more features** to test along the path to that node
- The DT starts to **overfit** (can check by monitoring validation set accuracy)





When to stop growing a Decision Tree

Important: You don't have to enforce **perfect purity**.

- It's okay to have a leaf node that is not 100% pure
  - Ex.  vs.
- By allowing a little bit of impurity, you can avoid growing the tree too much
- Predictions at Impure Leaf Nodes:
  - For test inputs that reach an impure leaf, we can predict the probability of belonging to each class
    - In above example, $p(red)$ = 3/8, $p(green)$ = 5/8
  - Or simply predict the majority label, which in this case would be green.

When to stop growing a Decision Tree

Stopping Criteria: The recursive splitting process continues until a stopping criterion is met.

Common stopping criteria:

- **Reaching a maximum depth** (preventing it from becoming overly complex).
- **Having a minimum number of samples per leaf** (splitting stops if a leaf node would contain fewer samples than a specified threshold).
- **Encountering pure nodes** (nodes containing only one class label).
  - or close-to-pure nodes, based on a predetermined threshold

Reasonable thresholds+criteria can be determined by doing cross-validation.

- Helps prevent stopping too early or too late

Avoiding Overfitting in DTs

Desired: a Decision Tree that is not too big/complex, yet fits the training data reasonably.

Method: Rather than preventing the DT from growing too complex, we can obtain a simpler tree by "**pruning**" it.

- Pruning is a technique where unnecessary branches/feature nodes are removed from the tree in order to improve performance.
  - The idea is to remove parts of the tree that are redundant or non-critical
  - Need good criteria for removal

Avoiding Overfitting in DTs

- Mainly two approaches to prune a complex DT:
  - **Pre-Pruning**: Prune while building the tree (stopping early)
  - **Post-Pruning**: Prune after building the tree
    - This is what most people are referring to when they say “pruning”
- Criteria for judging which nodes could potentially be pruned
  - Use a validation set (separate from the training set)
    - Prune each possible node that doesn’t hurt the accuracy on the validation set
    - Greedily remove the node that improves the validation accuracy the most
    - Stop when the validation set accuracy starts worsening

Pre-Pruning

Stop growing the tree at some point during construction, when it is determined that there is not enough data to make reliable choices.

Advantages:

- Prevents overfitting early on.
- Faster computation as the tree is smaller.

Disadvantages:

- May miss important patterns if stopped too early.
- Decision to stop may be subjective.

Post-Pruning

Grow the full tree and then prune (most popular).

- Allow tree to grow until best fit (allow overfitting)
- Then, remove nodes that seem not to have sufficient evidence or do not significantly contribute to the overall accuracy of the tree.
    - Subtrees are replaced with leaves

Advantages:

- Captures all possible patterns.
- Reduces overfitting by removing unnecessary nodes (improves accuracy).

Disadvantages:

- More computationally expensive initially.
- Risk of overfitting if not properly validated.

Post-Pruning Approaches

There are many techniques for pruning.

Some common ones:

- **Cost-Complexity Pruning**: Assign a cost to each subtree based on its accuracy (least error) and complexity, then choose the subtree with the lowest cost. (most popular)
- **Reduced Error Pruning**: Remove subtrees whose removal does not reduce accuracy / increase errors.
- **Minimum Impurity Decrease**: Remove subtrees whose Information Gain is below a chosen threshold (doesn't decrease impurity enough).
- **Minimum Leaf Size**: Remove leaf nodes with fewer training examples than a chosen threshold.

Example: A more complex DT in Python

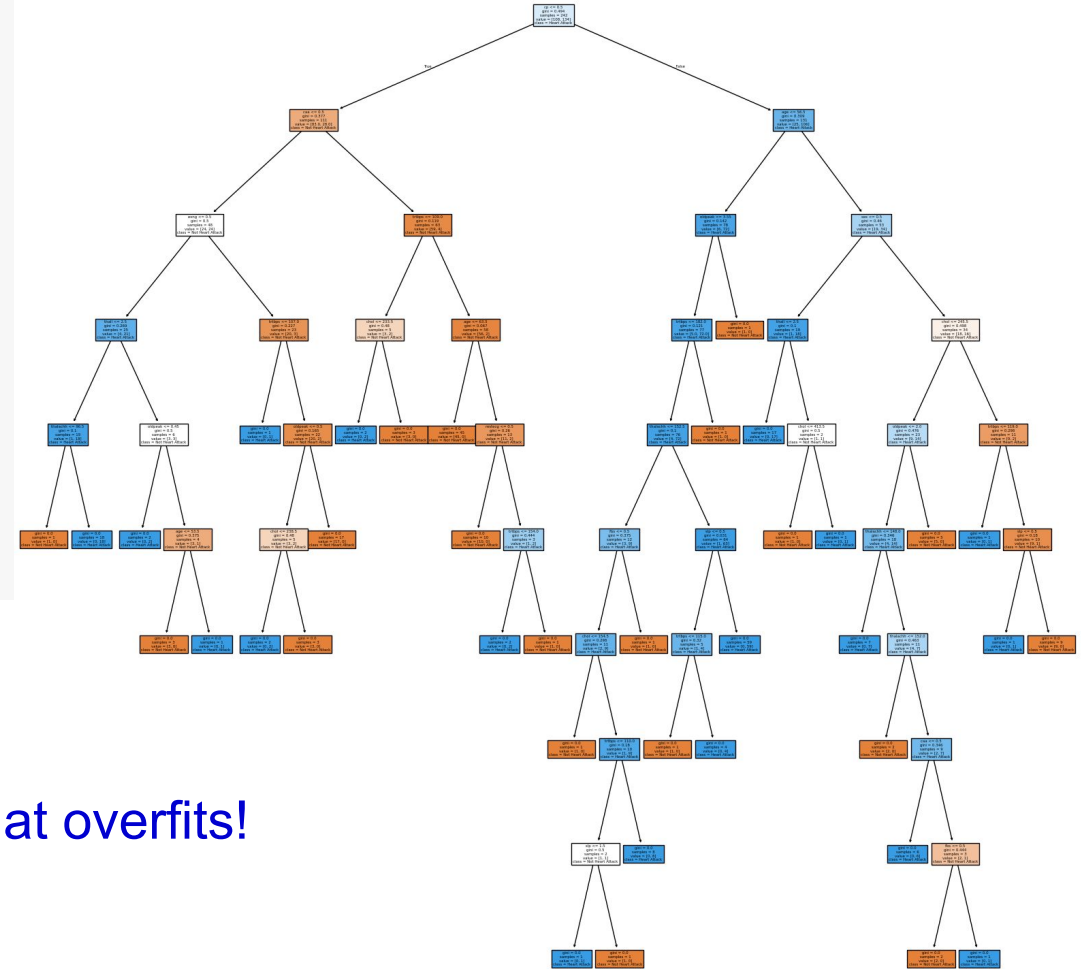
```
df = pd.read_csv('heart.csv')
X = df.drop('output', axis=1)
y = df['output']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=1)


model = tree.DecisionTreeClassifier(random_state=33)
model.fit(X_train,y_train)
y_pred = model.predict(X_test)

classes = ['Not Heart Attack', 'Heart Attack']
plt.figure(figsize=(20,20))
tree.plot_tree(model, feature_names=df.columns, class_names=classes,
               filled=True)
plt.show()
```

Complex Tree!



```
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)
print('Training Accuracy:', accuracy_score(y_train, y_train_pred))
print('Testing Accuracy:', accuracy_score(y_test, y_test_pred))
print('Tree Height:', model.tree_.max_depth)
```

That overfits!

```
Training Accuracy: 1.0
Testing Accuracy: 0.6557377049180327
Tree Height: 9
```

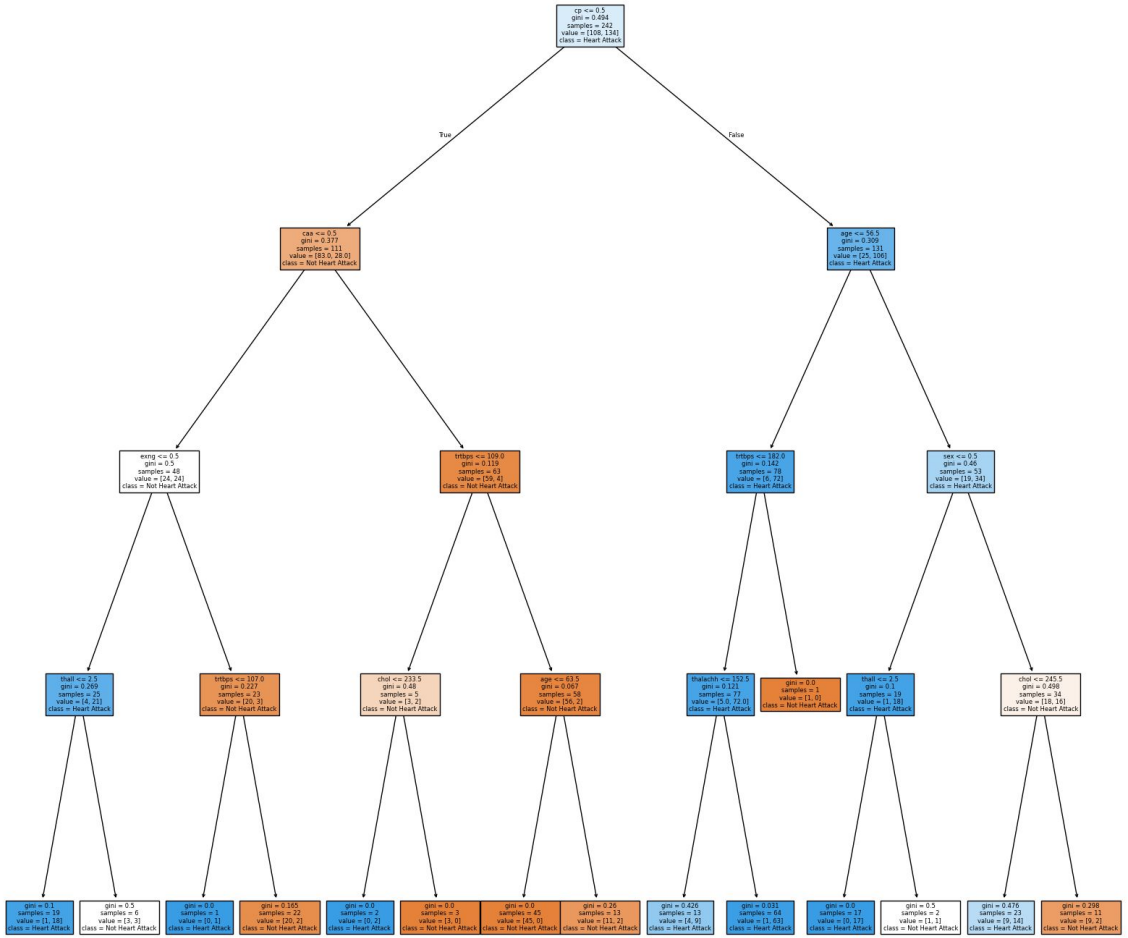
Example: A more complex DT in Python

How can we prune it?

Consider **Pre-Pruning**.

- We could set a **maximum depth** of the tree to avoid growing too deep

```
model = tree.DecisionTreeClassifier(random_state=33, max_depth=4)
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
```



Example: A more complex DT in Python

Pre-Pruning

- But how do we know what is a good number for our maximum depth?
- And what about setting a **minimum number of examples to split** a feature node?
- Or a **minimum number of examples for a leaf** node?

One way: use GridSearchCV to search for the best estimator using cross-validation.

```
from sklearn.model_selection import GridSearchCV

params = {'max_depth': [2, 4, 6, 8, 10],
          'min_samples_split': [4, 6, 8],
          'min_samples_leaf': [2, 4]}

model = tree.DecisionTreeClassifier()
gcv = GridSearchCV(estimator=model, param_grid=params)
gcv.fit(X_train, y_train)

print('Optimal parameters chosen:', gcv.best_params_)
```

```
Optimal parameters chosen: {'max_depth': 4, 'min_samples_leaf': 2, 'min_samples_split': 6}
```

Example: A more complex DT in Python

Consider **Post-Pruning**.

- We'll use the most common type, **Cost Complexity Pruning**.
  - **Cost** = Errors + Alpha*(# Nodes)
  - **Alpha**: fixed parameter that controls the tradeoff between complexity of the subtree and accuracy
    - Alpha = 0 → focus only on accuracy / reduction of errors
    - Large Alpha → put more emphasis on simplicity (more aggressive pruning)
    - Optimal alpha value minimizes the cost of the subtree (balance of accuracy and simplicity)
  - **Impurities**: higher impurity → higher uncertainty and heterogeneity

```
model = tree.DecisionTreeClassifier(random_state=42)

path = model.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas, impurities = path.ccp_alphas, path.impurities
```

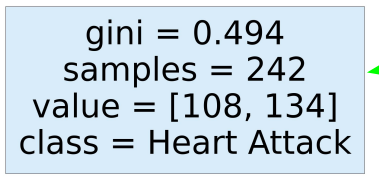
Example: A more complex DT in Python

- The **highest value of alpha** will **prune the entire tree**
  - Let's see what using the highest alpha looks like:

```
# Prune the entire tree:
clf = DecisionTreeClassifier(random_state=0, ccp_alpha=ccp_alphas[-1])
clf.fit(X_train, y_train)
print('# nodes in tree is:', clf.tree_.node_count)
print('Selected alpha: {0:0.2f}'.format(ccp_alphas[-1]))


classes = ['Not Heart Attack', 'Heart Attack']
plt.figure(figsize=(20,20))
tree.plot_tree(clf, feature_names=df.columns, class_names=classes,
               filled=True)
plt.show()
```

```
# nodes in tree is: 1
Selected alpha: 0.15
```



Entire tree is now just one node, and we're classifying everything as a heart attack!

→ Too aggressive.

Example: A more complex DT in Python

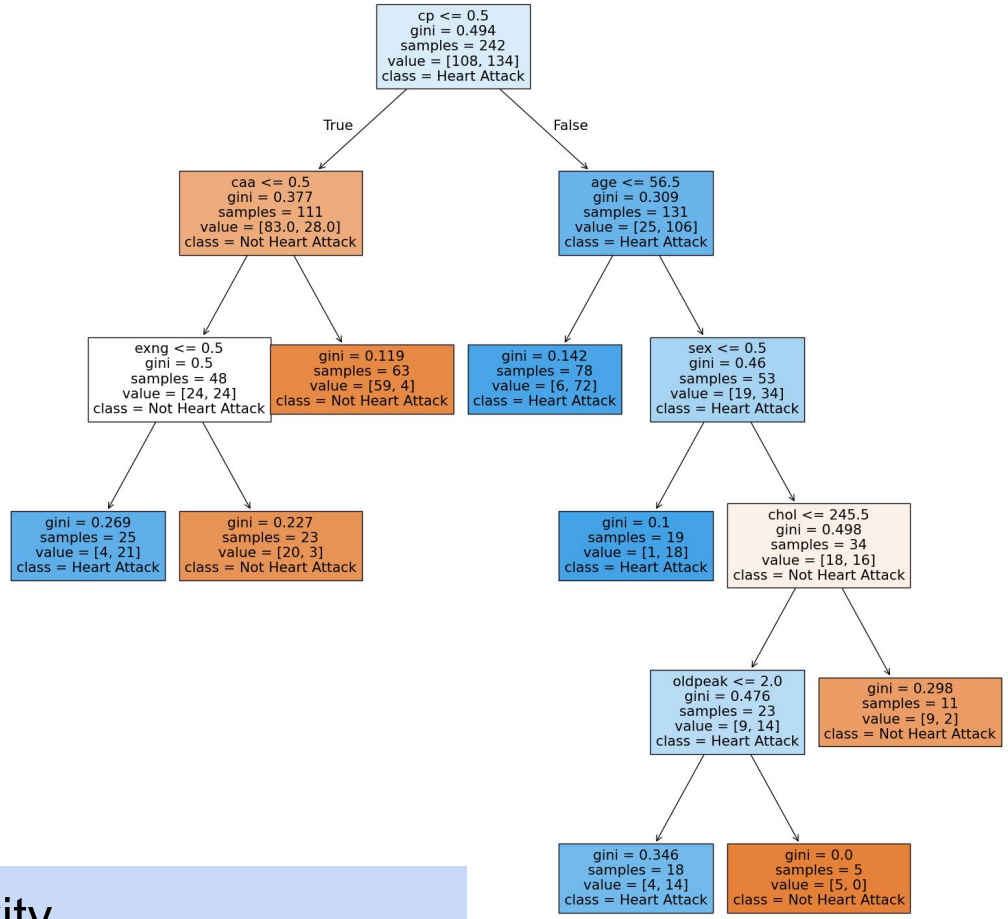
- Let's try alpha = 0.015

```
good_model = tree.DecisionTreeClassifier(random_state=33, ccp_alpha=0.015)
good_model.fit(X_train, y_train)
y_train_pred = good_model.predict(X_train)
y_test_pred = good_model.predict(X_test)

print('Training Accuracy:', accuracy_score(y_train, y_train_pred))
print('Testing Accuracy:', accuracy_score(y_test, y_test_pred))
print('Tree Height:', good_model.tree_.max_depth)

classes = ['Not Heart Attack', 'Heart Attack']
plt.figure(figsize=(20,20))
tree.plot_tree(good_model, feature_names=df.columns, class_names=classes,
               filled=True)
plt.show()
```

```
Training Accuracy: 0.9008264462809917
Testing Accuracy: 0.7049180327868853
Tree Height: 5
```



Decent balance of accuracy and tree complexity.
Still a difference between train and test accuracy, but less overfitting.

Utilizing DTs: From Training to Classification

To use a decision tree:

- First, go through the **training phase** where the tree decides on all its possible splits to build the tree.
- Then, use it for **classification**, where it is shown data with no label and it predicts the label it thinks is best.
    - (Note: DTs can also be used for Regression)

Decision Trees: A Summary

Key strengths:

- **Intuitive Decision Making**: Simple and easy to interpret and explain
- **Expressiveness**: Can represent any boolean function
- **Flexible**: Doesn't assume normality, can handle non-linear data, and easily handles different types of features (real, categorical, etc.), can handle missing values
- **Fast**: Very fast at test/prediction time
- **Adaptable**: Multiple DTs can be combined via ensemble methods (*future topic*)
  - More powerful than a single tree (ex. Decision Forests - *future topic*)
- **Usable**: Used in a number of real-world ML applications
  - ex. recommender systems, credit scoring, gaming, etc.

Decision Trees: A Summary

Key weaknesses:

- **Non-Optimal**: Learning an optimal DT is (NP-hard) intractable. Existing algorithms mostly rely on greedy heuristics.
- **Prone to Overfitting**: Can become very complex unless some pruning is applied.
- **Unstable**: Small variations in training data can substantially alter the resulting decision tree.
- **Bias**: Can be biased towards features with more levels, and when the training data is imbalanced tend to be biased toward the dominant class for classification.